

Análisis de Algoritmos

Divide y Vencerás, Algoritmos Codiciosos y Programación Dinámica

Aplicación a Procesamiento de Solicitudes de Servicio y Construcción de Redes Óptimas

Equipo de Programación
Año 2026

Tabla de Contenidos

- 1. Introducción
- 2. Descripción del Dataset
- 3. Módulo A — Divide y Vencerás
- 4. Módulo B — Algoritmo Codicioso
- 5. Módulo C — Programación Dinámica
- 6. Integración del Pipeline
- 7. Conclusiones
- 8. Herramientas Utilizadas
- 9. Referencias
- 10. Roles del Equipo

1. Introducción

Descripción del Problema

Este proyecto integra tres paradigmas fundamentales de análisis y diseño de algoritmos mediante su aplicación a un dataset real de telecomunicaciones. El objetivo es procesar 7,043 registros de clientes, extraer solicitudes de servicio, ordenarlas eficientemente, construir una infraestructura de red óptima y asignar recursos limitados de ancho de banda de forma óptima.

Contexto del Dataset

El dataset **WA_Fn-UseC_-Telco-Customer-Churn.csv** contiene información de clientes de una empresa de telecomunicaciones con 21 atributos por registro. De estos, extraemos:

- **customerID**: Identificador único de solicitud
- **tenure**: Antigüedad en meses (interpretado como prioridad)
- **MonthlyCharges**: Cargo mensual en USD
- **TotalCharges**: Cargo acumulado
- **Churn**: Estado activo (No) o en riesgo (Yes)

Justificación de Paradigmas

- **Módulo A (Divide y Vencerás)**: Requiere garantía $O(n \log n)$ en peor caso, estabilidad, y búsqueda eficiente.
- **Módulo B (Codicioso)**: La propiedad de elección codiciosa garantiza optimalidad global en MST.
- **Módulo C (DP)**: Subestructura óptima y contraejemplo demostrando falla del greedy.

2. Descripción del Dataset

Estadísticas del CSV Cargado

Métrica	Valor
Total de registros	7,043
Registros con TotalCharges nulo	11
Proporción de clientes activos (Churn = No)	~73.4%
Tenure máximo	72 meses
MonthlyCharges promedio	\$64.79 USD

Construcción del Grafo

El grafo se construye mediante agrupamiento sintético: los 7,043 clientes se distribuyen en **20 nodos** (regiones sintéticas). El costo de cada arista es $\blacksquare M_u + M_v \blacksquare$, donde M es el cargo promedio de cada nodo.

Métrica	Valor
Nodos	20
Aristas totales	190
Costo promedio de arista	129.042 USD

3. Módulo A — Divide y Vencerás

Algoritmo MergeSort

MergeSort ordena por tenure descendente mediante divide y vencerás:

función mergeSortByTenureDesc(requests):

 si requests está vacío: retornar

función merge(arr, left, mid, right):

 leftArray = arr[left : mid+1]

 rightArray = arr[mid+1 : right+1]

 i, j, k = 0, 0, left

 mientras i < len(leftArray) Y j < len(rightArray):

 si leftArray[i].tenure >= rightArray[j].tenure:

 arr[k] = leftArray[i]; i += 1

 sino:

 arr[k] = rightArray[j]; j += 1

 k += 1

Análisis de Complejidad

Aspecto	Análisis
Tiempo Peor Caso	$O(n \log n)$
Espacio	$O(n)$
Estabilidad	Sí

Resultados de Consultas Binarias

Consulta	Valor k	customerID	Tenure
Q_A01	72	5248-YGIJN	72
Q_A02	60	5248-YGIJN	72
Q_A03	45	5248-YGIJN	72
Q_A04	30	5248-YGIJN	72
Q_A05	12	5248-YGIJN	72

4. Módulo B — Algoritmo Codicioso (Kruskal)

Algoritmo Kruskal con Union-Find

```
función kruskal(V, E):  
    MST_edges = []; mst_weight = 0  
    dsu = UnionFind(V)  
    para cada arista (u, v, w) en E (ordenadas por w):  
        si dsu.find(u) ≠ dsu.find(v):  
            dsu.unite(u, v)  
            MST_edges.agregar((u, v, w))  
            mst_weight += w  
        si len(MST_edges) == V - 1: romper  
    retornar (MST_edges, mst_weight)
```

Lema del Ciclo

Lema: Si el peso de una arista en un ciclo es estrictamente mayor que todas las demás aristas del ciclo, entonces no puede estar en un MST. **Aplicación:** En el subgrafo de nodos 0-4, el ciclo 0-1-2-0 contiene la arista (0,1)=129 con peso máximo, por tanto se descarta al construir el MST.

Resultados del MST

Métrica	Valor
Número de nodos	20
Número de aristas en grafo	190
Peso total del MST	2,393 USD
Aristas en MST	19

5. Módulo C — Programación Dinámica

Formulación de Mochila 0-1

Dado $W = 500$ unidades y 50 solicitudes activas (Churn='No'), el objetivo es maximizar el valor total sujeto a capacidad límite. La recurrencia es: $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$

Solución Óptima Encontrada

Métrica	Valor
Valor máximo	5,030 centavos (\$50.30)
Solicitudes seleccionadas	9 de 50
Ancho de banda utilizado	500/500 (100%)

Contraejemplo: Fallo del Algoritmo Codicioso

Un algoritmo codicioso que selecciona items por máximo ratio v/w quedará atrapado en máximo local. La programación dinámica explora todas las combinaciones posibles encontrando la solución global óptima.

Enfoque	Seleccionadas	Valor Total	¿Óptimo?
Codicioso (ratio v/w)	5248-YGIJN	903	No
PD (Mochila 0-1)	5954-BDFSG	1,075	Sí

Pseudopolinomialidad

Complejidad: $O(n \cdot W) = O(50 \times 500) = O(25,000)$ operaciones. ¿Polinomial estricto? NO. Depende del VALOR de W , no de su representación binaria. Si $W = 2^k$, entonces $O(n \cdot W) = O(n \cdot 2^k)$ es exponencial en bits. Por tanto es PSEUDOPOLINOMIAL, viable en práctica pero no en teoría computacional.

6. Integración del Pipeline

Flujo de Datos

CSV (7,043 registros)



Módulo A: Parse + MergeSort



Arreglo ordenado por tenure

■ → Módulo B: Construcción MST (independiente)

■ → Módulo C: DP Mochila 0-1 (usa primeras 50 activas)

Dependencia A→C

Módulo C recibe el arreglo ordenado por tenure descendente de Módulo A. Esto asegura que las primeras 50 clientes activos sean los de mayor antigüedad, maximizando prioridad en asignación de ancho de banda.

7. Conclusiones

Comparación de Paradigmas

Aspecto	Divide y Vencerás	Codicioso	DP
Optimalidad	N/A	Sí (MST)	Sí
Tiempo	$O(n \log n)$	$O(E \log E)$	$O(n \cdot W)$
Espacio	$O(n)$	$O(V)$	$O(n \cdot W)$
En Datos Reales	Estable	Eficiente	Preciso

Hallazgos Clave

- En datos reales, los clientes activos de mayor tenure dominan el valor.
- MergeSort garantiza esta priorización mediante ordenamiento estable.
- Kruskal produce MST balanceado aprovechando la estructura de costos.
- DP asigna óptimamente recursos limitados donde greedy falla.

8. Herramientas Utilizadas

- **Lenguaje:** C++17
- **Compilador:** g++ (GCC 14.1.0) con flags -std=c++17 -O2
- **Librerías:** STL estándar (<vector>, <algorithm>, <chrono>)
- **Generación de código:** GitHub Copilot (asistencia verificada)
- **Control de versiones:** Git
- **Documentación:** Python + ReportLab para PDF

Nota: IA fue herramienta de productividad, no reemplazo de comprensión.

9. Referencias

Dataset: Kaggle - 'Telco Customer Churn' (CC0 Public Domain)

Cormen et al. (2009): Introduction to Algorithms, 3rd Ed. MIT Press

- Capítulo 4: Divide-and-Conquer
- Capítulo 23: Minimum Spanning Trees
- Capítulo 15: Dynamic Programming

Kleinberg & Tardos (2005): Algorithm Design. Pearson

Tarjan (1975): 'Efficiency of a good but not linear set union algorithm.' JACM 22(2)

10. Roles del Equipo

Rol	Responsabilidades	Módulos
Diseño	Formulación matemática, análisis de complejidad	A, B, C
Implementación A	Parser, MergeSort, BinSearch	A
Implementación B	Kruskal, Union-Find, grafo	B
Implementación C	DP, Backtracking, contraejemplo	C
Integración	Pipeline, compilación, testing	A, B, C
Documentación	Informe técnico, pseudocódigo	Todos

Fin del Informe Técnico
Generado: 03 de May de 2026
Versión: 1.0